

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

**Использование бинарных решающих диаграмм
для решения логических задач
по дисциплине «Математическая логика»**

Выполнил
студент гр. 5130904/30105

Ананьев А.М.

Руководитель

Шошмина И. В.

Санкт-Петербург
2025

Оглавление

Введение	3
Постановка задачи	4
Решение задачи	5
Соседи "northeast_neighbors" и "southeast_neighbors"	5
Библиотека dd	5
Интерпретация на естественном языке	8
Свойства и их значения	8
Заданные ограничения	9
Результат без склейки	10
Результат со склейкой	12
Заключение	14
Список литературы	15

Введение

Данная курсовая работа посвящена задаче поиска всех возможных решений булевой функции. Такая задача достаточно часто встречается в так называемых «логических» задачах, т.е. в задачах, в которых задано несколько ограничений и решение находится в области, где все ограничения истинны, например известная логическая задача Эйнштейна. Для поиска решений будет использоваться упорядоченная редуцированная бинарная решающая диаграмма (BDD).

Использование не традиционных форм представления булевой функции обоснованно тем, что «решение такой задачи в общем случае экспоненциально зависит от количества двоичных переменных, необходимых для ее формализации».[1]

Основная цель работы — построить по индивидуальному варианту измененную формализацию, программу и найти решение, а также построить интерпретацию на естественном языке. Для программирования задачи с использованием BDD будет использоваться язык Python с библиотекой `dd`, которая позволяет задавать ограничения различных типов и выводить пользователю найденные модели.

Постановка задачи

В основе курсовой работы лежит модифицированная версия задачи Эйнштейна.

Условия задачи:

- Имеется система, состоящая из 9 объектов, каждый из которых обладает 4 свойствами
- Каждое свойство может принимать 9 различных значений
- У двух различных объектов значения каждого свойства не совпадают

Цели:

- Найти все возможные решения
- Разработать физическую интерпретацию задачи
- Реализовать программу на Python с использованием библиотеки dd

Параметры варианта:

- $A = 1$
- $B = 30105$
- $C = 3$
- $v1 = 15$
- $v2 = 1$
- $v3 = 2$

V1			
15			*
		0	
			*

Таблица 1 Вариант шаблона для соседских отношений

V2	
1	Левая-правая граница

Таблица 2 Вариант склейки

V3	N1	N2	N3	N4
2	7	4	2	6

Таблица 3 Вариант с количеством ограничений

Решение задачи

В рамках данной работы объекты занимают ячейки на квадрате 3×3 с нумерацией от 0 до 8 (слева направо, построчно).

Отношения между объектами определяются через соседство, формализованное специализированными ограничивающими функциями.

Все условия задачи кодируются в виде булевых функций $p[k][i][j]$, где:

- k — идентификатор свойства
- i — идентификатор объекта
- j — значение свойства

Соседи "northeast_neighbors" и "southeast_neighbors"

Соседом объекта «А» является объект «Б» расположенный «справа-сверху» или «справа-снизу». Однако для первой и последней строки соседи могут располагаться, только на центральной строке, даже со склейкой. Без склейки у крайнего правого столбца нет соседей. Со склейкой у крайнего правого объекта в первой и последней строке появляется один сосед на центральной строке, первом столбце, а также у центрального крайнего правого объекта появляется два новых соседа: первый столбец верхняя и нижняя строка.

Соседи без склеек:

$\text{northeast_neighbors} = [(3, 1), (4, 2), (6, 4), (7, 5)]$

$\text{southeast_neighbors} = [(0, 4), (1, 5), (3, 7), (4, 8)]$

Соседи со склейками:

$\text{northeast_neighbors} = [(3, 1), (4, 2), (5, 0), (6, 4), (7, 5), (8, 3)]$

$\text{southeast_neighbors} = [(0, 4), (1, 5), (2, 3), (3, 7), (4, 8), (5, 6)]$

Первое число кортежа это – относительно какого объекта производится поиск соседа, второе число – сосед.

X	X	X	X	X	X	X
...	2	0	1	2	0	...
...	5	3	4	5	3	...
...	8	6	7	8	6	...
X	X	X	X	X	X	X

Таблица 4 Поле соседства

В таблице 4 отображены номера объектов, белые ячейки – поля к которым применяются правила соседства без склеек, синие ячейки отображают склейку «левая-правая граница», а красные ячейки показывают, где никогда не может быть соседей.

Библиотека dd

«dd — это пакет для работы с бинарными диаграммами решений, который включает в себя как

реализацию на чистом Python, так и привязки Cython к библиотекам C (CUDD, Sylvan, BuDDy). Модули Python и Cython реализуют один и тот же API, поэтому один и тот же пользовательский код работает с обоими. Доступны все стандартные операции с бинарными диаграммами решений, включая динамическое изменение порядка переменных с помощью просеивания, сборку мусора, выгрузку/загрузку из файлов, построение графиков и анализатор квантифицированных логических выражений.» [2]

В нашей программе мы импортируем *dd.autoref* [2] - это высокоуровневый интерфейс для реализации на чистом Python. В *autoref* [2] функциональный класс представляет узел. Все реализации используют обратные ребра, поэтому логическое отрицание занимает постоянное время.

Для создания *bdd*, во-первых, необходимо создать экземпляр менеджера и объявить переменные:

```
import dd.autoref as _bdd
```

```
bdd = _bdd.BDD()
```

```
bdd.declare('x', 'y', 'z')
```

(Binary Decision Diagrams (BDDs) in pure Python
and Cython wrappers of CUDD, Sylvan, and
BuDDy, б.д.)

В нашем случае необходимо объявить все 9 свойств, каждого из 4-х типов для 9-ти объектов:

```
var_names = []
```

```
for i in range(N):
```

```
    for k in range(M):
```

```
        for bit in range(T):
```

```
            var_name = f'x_{i}_{k}_{bit}'
```

```
            var_names.append(var_name)
```

```
bdd.declare(*var_names)
```

Далее для создания узлов можно вызвать *parser* [2], а именно *bdd.add_expr* [2], но я использую альтернативный метод, я создаю логическую константу TRUE - *bdd.true* [2] и после этого создаю узел для определенной переменной с помощью *bdd.var* [2]:

```
def encode_p(property, factory, value):
    if value < 0 or value >= N:
        return bdd.false
    expr = bdd.true
    for bit in range(T):
        var = var_name(factory, property,
bit)
        if (value >> bit) & 1:
            expr = expr & bdd.var(var)
        else:
            expr = expr & ~bdd.var(var)
    return expr
```

Далее по аналогии применяются ограничения.

Для применения логических операций к двум BDD-выражениям используется *bdd.apply*.

Для получения решений используется *bdd.pick_iter*, который создает итератор, перебирающий все возможные решения.

Интерпретация на естественном языке

Имеется завод, на территории которого располагается 9 цехов. Каждый цех собирает автомобили из разных стран, разных компаний, разных мощностей, разных цветов.

Свойства и их значения

- Страна производства (COUNTRY):
 - Япония
 - Германия
 - Франция
 - Китай
 - Корея
 - Италия
 - Англия
 - США
 - Россия
- Компания производитель (COMPANY):
 - Honda
 - Porsche
 - Peugeot
 - Geely
 - Kia
 - Ferrari
 - Jaguar
 - Ford
 - Lada
- Мощность (POWER):
 - 100
 - 110
 - 130
 - 170
 - 250
 - 410
 - 730
 - 850
 - 955
- Цвет (COLOR):
 - Красный

- Зеленый
- Синий
- Белый
- Черный
- Бежевый
- Фиолетовый
- Розовый
- Желтый

Заданные ограничения

Ограничения первого типа:

1. Общие
 - 1.1. Цех 0 - Страна Япония
 - 1.2. Цех 1 - Страна Франция
 - 1.3. Цех 4 - Мощность 250
 - 1.4. Цех 8 - Цвет Желтый
 - 1.5. Цех 2 - Компания Ferrari
 - 1.6. Цех 2 - Цвет Белый
 - 1.7. Цех 1 - Мощность 170
2. Только со склейкой
 - 2.1. Цех 0 - Цвет Зеленый

Ограничения второго типа:

1. Общие
 - 1.1. Англии соответствует 410 л.с.
 - 1.2. США соответствует Цвет Синий
 - 1.3. Корею соответствует KIA
 - 1.4. 130 л.с. соответствует Цвет Черный
 - 1.5. Geely соответствует 100 л.с.
 - 1.6. Jaguar соответствует Цвет Зеленый
2. Только без склейки
 - 2.1. Peugeot соответствует 110 л.с.

Ограничения третьего типа:

1. Общие
 - 1.1. Lada находится юго-восточнее Франция
 - 1.2. Розовый находится северо-восточнее KIA
 - 1.3. Ford находится северо-восточнее Германия
 - 1.4. Peugeot находится юго-восточнее Китай
2. Только без склейки

2.1. 100 л.с. находится юго-восточнее Красный

3. Только со склейкой

3.1. Китай находится юго-восточнее 410 л.с.

Ограничения четвертого типа:

1. Общие

1.1. Корея соседствует с Цвет Бежевый

1.2. 130 л.с. соседствует с Цвет Фиолетовый

1.3. Китай соседствует с Япония

1.4. Россия соседствует с США

1.5. Цвет Красный соседствует с Geely

1.6. 100 л.с. соседствует с 850 л.с.

2. Только со склейкой

2.1. Jaguar соседствует с 850 л.с.

Результат без склейки

--- Решение 1 ---

Япония	Honda	130	Черный	Франция	Porsche	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	955	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Jaguar	730	Зеленый	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 2 ---

Япония	Honda	130	Черный	Франция	Porsche	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	730	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Jaguar	955	Зеленый	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 3 ---

Япония	Honda	130	Черный	Франция	Porsche	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	850	Красный	Китай	Ford	250	Фиолетовый	США	Lada	955	Синий	
Германия	Jaguar	730	Зеленый	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 4 ---

Япония	Honda	130	Черный	Франция	Porsche	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	850	Красный	Китай	Ford	250	Фиолетовый	США	Lada	730	Синий	
Германия	Jaguar	955	Зеленый	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 5 ---

Япония	Jaguar	955	Зеленый	Франция	Honda	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	730	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 6 ---

Япония	Jaguar	955	Зеленый	Франция	Honda	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	850	Красный	Китай	Ford	250	Фиолетовый	США	Lada	730	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

--- Решение 7 ---

Япония	Jaguar	955	Зеленый	Франция	Honda	170	Розовый	Англия	Ferrari	410	Белый	
Корея	Kia	850	Красный	Китай	Ford	250	Фиолетовый	США	Lada	730	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Италия	Peugeot	110	Желтый	

Результат со склейкой

--- Решение 1 ---

Япония	Jaguar	955	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	730	Белый	
Корея	Kia	110	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 2 ---

Япония	Jaguar	955	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	110	Белый	
Корея	Kia	730	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 3 ---

Япония	Jaguar	955	Зеленый	Франция	Porsche	170	Розовый	Италия	Ferrari	730	Белый	
Корея	Kia	110	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Honda	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 4 ---

Япония	Jaguar	955	Зеленый	Франция	Porsche	170	Розовый	Италия	Ferrari	110	Белый	
Корея	Kia	730	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Honda	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 5 ---

Япония	Jaguar	730	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	955	Белый	
Корея	Kia	110	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 6 ---

Япония	Jaguar	730	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	110	Белый	
Корея	Kia	955	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 7 ---

Япония	Jaguar	730	Зеленый	Франция	Porsche	170	Розовый	Италия	Ferrari	955	Белый	
Корея	Kia	110	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Honda	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 8 ---

Япония	Jaguar	730	Зеленый	Франция	Porsche	170	Розовый	Италия	Ferrari	110	Белый	
Корея	Kia	955	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Honda	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 9 ---

Япония	Jaguar	110	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	955	Белый	
Корея	Kia	730	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	
Германия	Porsche	130	Черный	Россия	Geely	100	Бежевый	Англия	Peugeot	410	Желтый	

--- Решение 10 ---

Япония	Jaguar	110	Зеленый	Франция	Honda	170	Розовый	Италия	Ferrari	730	Белый	
Корея	Kia	955	Красный	Китай	Ford	250	Фиолетовый	США	Lada	850	Синий	

Германия Porsche 130 Черный	Россия Geely 100 Бежевый	Англия Peugeot 410 Желтый	
+-----+-----+-----+			
--- Решение 11 ---			
+-----+-----+-----+			
Япония Jaguar 110 Зеленый	Франция Porsche 170 Розовый	Италия Ferrari 955 Белый	
+-----+-----+-----+			
Корея Kia 730 Красный	Китай Ford 250 Фиолетовый	США Lada 850 Синий	
+-----+-----+-----+			
Германия Honda 130 Черный	Россия Geely 100 Бежевый	Англия Peugeot 410 Желтый	
+-----+-----+-----+			
--- Решение 12 ---			
+-----+-----+-----+			
Япония Jaguar 110 Зеленый	Франция Porsche 170 Розовый	Италия Ferrari 730 Белый	
+-----+-----+-----+			
Корея Kia 955 Красный	Китай Ford 250 Фиолетовый	США Lada 850 Синий	
+-----+-----+-----+			
Германия Honda 130 Черный	Россия Geely 100 Бежевый	Англия Peugeot 410 Желтый	
+-----+-----+-----+			

Всего найдено решений: 12

Заключение

В ходе выполнения работы была исследована задача логического вывода с множеством объектов и свойств, аналогичная классической задаче Эйнштейна. Основное внимание уделялось формализации условий задачи, их преобразованию в булевы формулы и эффективному поиску решений.

Была разработана программа на языке Python с использованием библиотеки dd, которая позволяет задавать ограничения четырех различных типов и находить все допустимые решения. Программа реализует параметризованный подход к определению соседских отношений с поддержкой двух режимов работы - со склейкой границ и без склейки.

В результате работы получены полные решения для системы из девяти цегов с четырьмя свойствами, каждое из которых принимает девять различных значений. Полученные результаты подтверждают практическую применимость методов булевых функций и двоичных диаграмм решений для анализа и решения сложных логических задач.

Список литературы

1. *Binary Decision Diagrams (BDDs) in pure Python and Cython wrappers of CUDD, Sylvan, and BuDDy*. (б.д.). Получено из GitHub: <https://github.com/tulip-control/dd>
2. Карпов, Ю. Г. (б.д.). *Курс / Модуль 4. Бинарные решающие диаграммы / Конспект модуля 4*. Получено из Математическая логика: https://apps.openedu.ru/learning/course/course-v1:spbstu+MATLOG+fall_2025/block-v1:spbstu+MATLOG+fall_2025+type@sequential+block@b066bb9741d844df83f03e323bf703e2/block-v1:spbstu+MATLOG+fall_2025+type@vertical+block@a62e15d3faf34df795506671018ae8af
3. Шошмина, И. В. (2024). *Методические рекомендации к курсовой работе по математической логике*. Санкт-Петербург: Издательство Политехнического университета.